Z_Übung – Tabellen verbinden mit JOIN

1000 Datenbank · Teil 3 SQL · Zusatzübungen · MySQL Workbench



Nonna will es genau wissen

„Welche Rezepte kommen gut an, welche Zutaten brauche ich am häufigsten und welche Leser sind besonders aktiv? Hilf mir, meinen Blog richtig auszuwerten!“

Ziel	Nonnas Tabellen mit INNER JOIN und LEFT JOIN verbinden und mit GROUP BY, HAVING und ORDER BY auswerten – anspruchsvollere Aufgaben als in Übung Z.
Startpunkt	`nonnas_blog.sql` (Ordner `material`) – legt die Datenbank `nonnas_blog` an; **frisch ausführen**, damit alle dieselben Daten haben
Tabellen	`kategorie` · `rezept` (rezept_id, titel, dauer_min, schwierigkeit, veroeffentlicht, kategorie_id) · `zutat` (zutat_id, name, einheit) · `rezept_zutat` (rezept_id, zutat_id, menge) · `leser` (leser_id, vorname, nachname, email) · `kommentar` (kommentar_id, rezept_id, leser_id, text, sterne, geschrieben)
Ablage	deine Lösungen in `datenbank/abfragen_nonna.sql` (mit Kommentar `-- Übung Z, Aufgabe …`)
Folien	Präsentation „1000 Datenbank“ – vor der Übung erklärt und vorgemacht; bei jeder Aufgabe steht die passende Folie
Tipp	Nonnas ER-Diagramm (Übung 02) zeigt die ON-Bedingungen: jede Linie = Fremdschlüssel = Primärschlüssel.

Aufgabe 1 – Rezepte mit Kategorie, Kommentaren und Bewertung

Hilfe: *Präsentation „1000 Datenbank“, Folie 0720–0730, 0750 (Nummer unten rechts)*

Titel, Kategorie (Bezeichnung), Anzahl der Kommentare und durchschnittliche Sternebewertung (auf eine Nachkommastelle gerundet) aller veröffentlichten Rezepte (Spalte `veroeffentlicht` ist nicht NULL) – **auch** Rezepte ohne Kommentar: dort sollen 0 Kommentare und die Bewertung 0 erscheinen (`IFNULL` oder `COALESCE`).

Sortiere nach der Anzahl der Kommentare absteigend, bei Gleichstand nach dem Titel aufsteigend.

```
USE nonnas_blog;
SELECT r.titel, k.bezeichnung AS kategorie,
       COUNT(c.kommentar_id) AS anzahl_kommentare,
       COALESCE(ROUND(AVG(c.sterne), 1), 0.0) AS bewertung
FROM rezept AS r
INNER JOIN kategorie AS k ON k.kategorie_id = r.kategorie_id
LEFT JOIN kommentar AS c ON c.rezept_id = r.rezept_id
WHERE r.veroeffentlicht IS NOT NULL
GROUP BY r.rezept_id, r.titel, k.bezeichnung
ORDER BY anzahl_kommentare DESC, r.titel ASC;
```

Die Abfrage liefert 5 veröffentlichte Rezepte. Tiramisu steht mit 3 Kommentaren und 4,7 Sternen zuerst. Bei Minestrone zählt COUNT 0 Kommentare, und COALESCE setzt die fehlende Durchschnittsbewertung auf 0,0.

Aufgabe 2 – Zutaten in mehreren Rezepten

Hilfe: *Präsentation „1000 Datenbank“, Folie 0680, 0740 (Nummer unten rechts)*

Name und Einheit jeder Zutat, die in mindestens 2 Rezepten vorkommt – dazu die Anzahl der Rezepte und die Gesamtmenge über alle Rezepte (`SUM(menge)`).

Sortiere nach der Anzahl der Rezepte absteigend, bei Gleichstand nach dem Namen der Zutat aufsteigend. Du brauchst `COUNT(DISTINCT …)` und `HAVING`.

```
SELECT z.name, z.einheit,
       COUNT(DISTINCT rz.rezept_id) AS anzahl_rezepte,
       SUM(rz.menge) AS gesamtmenge
FROM zutat AS z
INNER JOIN rezept_zutat AS rz ON rz.zutat_id = z.zutat_id
GROUP BY z.zutat_id, z.name, z.einheit
HAVING COUNT(DISTINCT rz.rezept_id) >= 2
ORDER BY anzahl_rezepte DESC, z.name ASC;
```

Knoblauch, Parmesan und Tomaten stehen in je 3 Rezepten. Ihre Gesamtmengen sind 5 Zehen, 190 g und 1500 g. Zucker kommt in 2 Rezepten vor und ergibt insgesamt 160 g. COUNT(DISTINCT …) zählt die verschiedenen Rezept-IDs.

Aufgabe 3 – Aktive Leser mit guter Laune

Hilfe: *Präsentation „1000 Datenbank“, Folie 0740, 0750 (Nummer unten rechts)*

Vollständiger Name der Leser (Vor- und Nachname in einer Spalte, `CONCAT`), Anzahl ihrer Kommentare und ihre durchschnittliche Sternebewertung – nur Leser mit mindestens 2 Kommentaren **und** einem Durchschnitt von mindestens 4 Sternen.

Sortiere nach dem Durchschnitt absteigend, danach nach der Anzahl der Kommentare absteigend, danach nach dem Namen aufsteigend. Verbinde `leser` und `kommentar`.

```
SELECT CONCAT(l.vorname, ' ', l.nachname) AS lesername,
       COUNT(*) AS anzahl_kommentare,
       AVG(c.sterne) AS durchschnitt
FROM leser AS l
INNER JOIN kommentar AS c ON c.leser_id = l.leser_id
GROUP BY l.leser_id, l.vorname, l.nachname
HAVING COUNT(*) >= 2 AND AVG(c.sterne) >= 4
ORDER BY durchschnitt DESC, anzahl_kommentare DESC, lesername ASC;
```

Daisy Sarasa und Rosalina Stella haben je 2 Kommentare und im Mittel 4,5 Sterne vergeben. Peach Toadstool hat 2 Kommentare und einen Durchschnitt von 4,0. HAVING prüft beide Bedingungen pro Leser.

Aufgabe 4 – Aufwendige Rezepte mit Zutatenliste

Hilfe: *Präsentation „1000 Datenbank“, Folie 0740 (Nummer unten rechts)*

Titel, Kategorie, Zubereitungsdauer (`dauer_min`), Anzahl der unterschiedlichen Zutaten und eine Spalte mit allen Zutatennamen in einer Zeile, durch Komma getrennt (`GROUP_CONCAT(… SEPARATOR ', ')`).

- Nur Rezepte mit mindestens 3 unterschiedlichen Zutaten.
- Nur Rezepte mit einer Zubereitungsdauer von mehr als 20 Minuten.
- Sortiere nach der Anzahl der Zutaten absteigend, danach nach der Dauer aufsteigend.

Du brauchst vier Tabellen: `rezept`, `kategorie`, `rezept_zutat` und `zutat`.

```
SELECT r.titel, k.bezeichnung AS kategorie, r.dauer_min,
       COUNT(DISTINCT z.zutat_id) AS anzahl_zutaten,
       GROUP_CONCAT(z.name ORDER BY z.name SEPARATOR ', ') AS zutatenliste
FROM rezept AS r
INNER JOIN kategorie AS k ON k.kategorie_id = r.kategorie_id
INNER JOIN rezept_zutat AS rz ON rz.rezept_id = r.rezept_id
INNER JOIN zutat AS z ON z.zutat_id = rz.zutat_id
WHERE r.dauer_min > 20
GROUP BY r.rezept_id, r.titel, k.bezeichnung, r.dauer_min
HAVING COUNT(DISTINCT z.zutat_id) >= 3
ORDER BY anzahl_zutaten DESC, r.dauer_min ASC;
```

Tiramisu hat 6 Zutaten und dauert 30 Minuten. Danach folgen Risotto ai Funghi (45 Minuten), Minestrone (60 Minuten) und Lasagne (90 Minuten) mit je 4 Zutaten. GROUP_CONCAT schreibt die Zutatennamen in eine gemeinsame Spalte.

Aufgabe 5 – Die beliebteste Kategorie

Hilfe: *Präsentation „1000 Datenbank“, Folie 0740, 0750 (Nummer unten rechts)*

Welche Kategorie hat die beste durchschnittliche Sternebewertung? Zeige Bezeichnung, Anzahl der Kommentare und Durchschnitt – nur Kategorien mit mindestens 2 Kommentaren, und nur die **eine** beste Zeile (`ORDER BY … LIMIT 1`). Du brauchst `kategorie`, `rezept` und `kommentar`.

```
SELECT k.bezeichnung AS kategorie,
       COUNT(c.kommentar_id) AS anzahl_kommentare,
       AVG(c.sterne) AS durchschnitt
FROM kategorie AS k
INNER JOIN rezept AS r ON r.kategorie_id = k.kategorie_id
INNER JOIN kommentar AS c ON c.rezept_id = r.rezept_id
GROUP BY k.kategorie_id, k.bezeichnung
HAVING COUNT(c.kommentar_id) >= 2
ORDER BY durchschnitt DESC
LIMIT 1;
```

Dessert gewinnt mit 4 Kommentaren und einem Durchschnitt von 4,75 Sternen. HAVING prüft die Mindestanzahl. LIMIT 1 liefert nach der Sortierung die beste Kategorie.

Das kann ich jetzt

- ☐ Ich verbinde drei und vier Tabellen mit INNER JOIN.
- ☐ Ich kombiniere JOIN mit GROUP BY, HAVING und mehreren Aggregatfunktionen.
- ☐ Ich weiß, wann ich LEFT JOIN brauche und wie ich NULL-Werte ersetze.

Zusatzaufgaben

- Welche Leser haben noch nie einen Kommentar geschrieben? (LEFT JOIN … IS NULL)

```
SELECT l.vorname, l.nachname
FROM leser AS l
LEFT JOIN kommentar AS c ON c.leser_id = l.leser_id
WHERE c.kommentar_id IS NULL;
```

Yoshi Isola hat noch keinen Kommentar geschrieben. Für ihn liefert der LEFT JOIN eine Zeile mit NULL in kommentar_id.

- Welche Rezepte dauern länger als der Durchschnitt ihrer Kategorie? (korrelierte Unterabfrage)

```
SELECT r.titel, r.dauer_min
FROM rezept AS r
WHERE r.dauer_min > (
    SELECT AVG(r2.dauer_min)
    FROM rezept AS r2
    WHERE r2.kategorie_id = r.kategorie_id
)
ORDER BY r.rezept_id;
```

Tiramisu (30 Minuten), Lasagne (90 Minuten) und Minestrone (60 Minuten) liegen über dem Durchschnitt ihrer Kategorie. Die Unterabfrage nutzt die Kategorie des äußeren Rezepts.

- Lege mit `CREATE VIEW rezept_statistik AS …` eine Sicht mit Titel, Kategorie, Anzahl Kommentare und Durchschnitt an (Aufgabe 1) und frage nur die Rezepte mit mehr als 4 Sternen ab.

```
CREATE VIEW rezept_statistik AS
SELECT r.titel, k.bezeichnung AS kategorie,
       COUNT(c.kommentar_id) AS anzahl_kommentare,
       COALESCE(ROUND(AVG(c.sterne), 1), 0.0) AS bewertung
FROM rezept AS r
INNER JOIN kategorie AS k ON k.kategorie_id = r.kategorie_id
LEFT JOIN kommentar AS c ON c.rezept_id = r.rezept_id
WHERE r.veroeffentlicht IS NOT NULL
GROUP BY r.rezept_id, r.titel, k.bezeichnung;

SELECT titel, kategorie, anzahl_kommentare, bewertung
FROM rezept_statistik
WHERE bewertung > 4
ORDER BY bewertung DESC, titel ASC;
```

Die Sicht verwendet dieselbe Auswertung wie Aufgabe 1. Panna Cotta mit 5,0 Sternen und Tiramisu mit 4,7 Sternen erfüllen die Bedingung bewertung > 4.

Zum Schluss: sichern mit Git

Speichere alles (Strg + S) und sichere deinen Stand im Repository `marios-pizza`:

```
git add .
git commit -m "SQL: Tabellen verbinden mit Nonna's Blog (Übung Z)"
git push
```

Kontrolle: auf github.com das Repository neu laden – dein Commit und die neuen Dateien sind zu sehen.